One-hour Jenkins challenge: investigate red, earn green

Your mission

The supplied app compiles but contains a bug. Its tests are intentionally failing. Use Jenkins to discover what is wrong, fix the application, and prove that your next Git commit produces a green pipeline. Do not add features, write new tests, or change the 28 supplied tests. The starter also has no SCM trigger: configuring that is your task.

Time budget: 60 minutes after environment preparation. JDK/Git, Jenkins/plugins, the `student-laptop` agent, and the local Git server must be ready before starting the clock. A green app build is NOT a prerequisite. Ask the instructor for help if setup fails before the Test stage.

Before the first build

Your alias is read automatically from the author of the Git commit Jenkins checks out, not from a properties entry. Check your project identity:

```
git config user.name
git log -1 --format=%an
```

The first command shows your name for future commits; the second shows the existing commit's author, which the pipeline records. Changing Git config does not rewrite old commits. If needed, use `git config user.name "YourAlias"` in this repository and make a new commit. No author email is recorded. Use an instructor-approved pseudonym if you do not want your real name in the evidence, and let the instructor know the Git author name you will use.

The instructor gives you two non-secret values for `submission.properties`:

- `class.code`: the same class code used by everyone in your group.

- `challenge.code`: your individual instructor-issued submission code.

Commit this file. Do not add a `student.alias` property; Git supplies it. The pipeline records build evidence described below; read that notice first. No new software is needed. Use a new Jenkins job, not an already-configured one.

Suggested plan

Time	Your work
0-15 min	Create the SCM job; enter your issued codes and configure polling.
15-25 min	Run once manually; inspect the expected test failure.
25-35 min	Personalize the help introduction; commit and observe automatic red.
35-50 min	Diagnose the failing assertions, fix the app, commit, and observe automatic green.
50-60 min	Download/run the JAR and submit red/green evidence.

1. Connect Jenkins to your repository

Create a **Pipeline** job named `festpass-ci` (use a new name if it exists). Choose **Pipeline script from SCM**, then **Git**:

- Repository: `git://127.0.0.1:9418/festpass-java`

- Credentials: none; branch: `*/main`; script path: `Jenkinsfile`.

Use the URL printed by the local Git launcher if your port differs. Keep the Git server, Jenkins, and agent terminals open.

Add a top-level `triggers` block to the original repository's Jenkinsfile, inside `pipeline`, beside `options` and `stages`. Use `pollSCM` to check Git once per minute. Guide section 7 provides a syntax example. Commit both your trigger and submission settings before the first run:

```
git add Jenkinsfile submission.properties
git commit -m "Configure my Jenkins assignment"
```

Save the job and select **Build Now once**. This bootstrap run loads your committed trigger. Expect the **Test stage to fail**, with failed tests visible in Test Result and Console Output. Checkout, Submission setup, and compilation must succeed; a missing plugin or invalid submission code is not the intended bug.

Polling builds when Git changes; Build periodically runs on a timer regardless of changes. Do not use the latter or configure a duplicate UI trigger.

2. Prove an automatic build, still red

Change only the introductory message in the app's `help` output in `src/main/java/FestPass.java`, adding your Git author alias. Keep command names unchanged and do not fix the test defect yet.

```
git add src/main/java/FestPass.java
git commit -m "Personalize my help introduction"
git log -1 --oneline
```

Do NOT press Build Now. Wait for polling (usually 1-2 minutes). The automatic build should still fail its tests. Capture its cause, **Started by an SCM change**, and compare its checked-out commit with Git. Wait for this build to finish before committing again so polling sees it. This local Git-server lab needs a commit, not a push; saving a file is not enough.

3. Investigate and fix

Open a failed test in Jenkins. Read its expected behavior, actual result, and the related application code. Explain the violated business rule in your own words. Make the smallest application fix you believe addresses the failures. The assignment deliberately does not tell you which expression to change.

Run `java ci/Build.java verify` locally, then `java ci/SmokeTest.java`. Commit your fix with a meaningful message. Without clicking Build Now, observe Jenkins turn **SUCCESS with 28 tests, 0 failures**. If it stays red, investigate and repeat rather than disabling a check.

Leave tests, build helpers, evidence code, and pipeline stages intact. Do not delete assertions, ignore exit codes, skip tests, hardcode test inputs, or change the expected answers. Adding the polling trigger is the only required Jenkinsfile change.

4. Check the products and evidence

The failed run must skip Build JAR, Smoke test JAR, and Save artifact. It must publish **no JAR**. It WILL publish submission-evidence.csv: that diagnostic evidence is intentionally archived even when tests fail.

Download the final successful run's JAR and evidence CSV. In its download folder, run `java -jar festpass.jar help` and show your personalized message. Save each evidence CSV in a separate build folder so downloads do not overwrite.

Build evidence notice

Every configured run records the checked-out commit's author name as your alias, issued codes, commit ID, job/build number, UTC time, result, test counts, source/pipeline hashes, and the successful JAR hash. It also records a random per-installation ID outside the Git workspace and a class-scoped hash of the computer name. No raw computer name, username, IP, MAC address, or hardware serial is included.

The installation ID is stored under the build agent user's home in `.festpass-assessment/<class.code>/installation-id.txt`. Do not copy or reset it between builds. Keep it until grading is complete. These are pseudonymous assessment records, not hardware identity certificates. Tell the instructor if you use a shared lab machine or need an approved live-demo alternative. A short demonstration may be part of assessment.

Grading rubric (100 points)

Area	Points	Full-credit evidence
Job configuration	20	Correct SCM URL, branch, Jenkinsfile path, and agent.
Trigger and commit trace	25	Committed polling trigger (15); automatic red build and matching commit/cause (10).
Diagnosis and repair	30	Explain a failed assertion/business rule (15); minimal fix with unchanged tests (15).
Green build and artifacts	15	Automatic 28-test green build (10); failed-run JAR gating and working final JAR (5).
Evidence and explanation	10	Correct issued codes, red/green evidence CSVs, and brief explanation/live demonstration.

Submit your source/repository, red and green evidence CSVs, the final JAR, and a short evidence page. Include build numbers, commit IDs, build causes, one failing assertion, skipped product stages, successful test count, and JAR output. Briefly explain what you changed and why; no long report. Never submit Jenkins home, passwords, agent secrets, or real student data.